Recovery-on-HARVEST: A Manipulation-Data Platform, and a Learned Selector for *Which* Recovery a Failed Policy Should Run

Vignan Kamarthi
RIVeR Lab, Northeastern University

Project Proposal

Thesis. Project HARVEST equips the RIVeR Lab to study condition-aware robotic manipulation of donated canned goods, a real food-bank bottleneck with asymmetric safety stakes. I propose two layers on one platform. First, build the HARVEST framework and use it to collect a real, physical multimodal dataset of condition-aware can manipulation on the Kinova, the centerpiece of the project. I develop the framework simulation-first so the recording pipeline is complete and validated before the robot is committed, but the goal is the physical data. Second, layer on top of it my research contribution, making recovery-strategy *selection* a learnable, cost-aware decision problem, so that when the manipulation policy fails on a damaged can, a small learned selector chooses a recovery that aims to beat any fixed single-mechanism baseline at matched intervention budget. The failures the policy makes during HARVEST data collection are not noise to discard. They are exactly the substrate the recovery layer consumes, so the research rides on the platform at zero marginal data cost.

1. Motivation and the HARVEST platform

Food banks inspect thousands of heterogeneous donated cans per shift, a task performed by volunteers whose availability is inherently fragile. The inspection is safety-critical and non-uniform across damage types. A sharp dent on a top or side seam can damage the seam and admit bacteria, a bulging or swollen can is a tell-tale sign of botulism (*Clostridium botulinum*) and must be discarded, and a shallow body dent is typically safe (per USDA and CDC food-safety guidance). Sorting thousands of such items by hand is repetitive and error-prone, and volunteer labor is fragile and hard to staff, which motivates a consistent, autonomous, condition-aware alternative that works at any hour.

The HARVEST platform collects a multimodal dataset of condition-aware pick-and-reorient of cans. The robot grasps a can of unknown orientation and condition, reorients it to expose the nutrition label for overhead verification, and the system records every sensor stream time-aligned, across hundreds of cans and thousands of episodes. That physical dataset is the goal. To get there efficiently, I develop the framework simulation-first, where a physics simulation (arm and can) generates episodes that exercise and validate the full recording pipeline before any robot time. A modular, decoupled, and highly cohesive software design keeps every external dependency (the simulator, each sensor modality, the on-disk format, the base policy) behind an interface, so the real Kinova Gen3 drops in by swapping a single backend and physical collection begins. The design rests on three commitments.

- **Sensor suite** (full, with each stream marked hardware-contingent). An overhead RGB-D camera, static and top-down, giving both color and depth of the whole workspace, where the depth is label-visibility ground truth and a rich geometric read of the scene, including the surface deformation that most directly distinguishes a bulged can. The Gen3 wrist module, also an RGB-D camera, giving close-up color and depth at the grasp itself as the arm moves. Tactile contact-pressure from the Robotiq TSF-85 fingertips, a 28-taxel fingertip kit rather than a gripper. Force-torque, taken by default from the Gen3 joint-torque sensors, which are always

present. If the lab's arm carries the optional wrist-mounted 6-axis force-torque sensor, we use that instead for a cleaner, direct reading, and each recorded stream notes which source produced it. And proprioception from the joint states.

- **A five-class condition taxonomy grounded in USDA safety criteria**. The classes run from nominal, through body dent (typically safe), seam dent (seal-compromising), and bulge or swelling (botulism sign), to rust or corrosion.
- **Honest data accounting**. The unit of statistical independence is the *can* (about 50 per class), not the episode, so splits are made *by can* and a can never appears in more than one of train, validation, and test. Grasp-stability labels come from simulator ground truth in simulation, and from an independent source such as overhead-vision success or a human on the real robot. They never come from the tactile stream, since that is the very signal a tactile ablation is meant to evaluate.

The static overhead camera is registered to the robot's coordinate frame once, through eye-to-hand extrinsic calibration with ArUco fiducial markers, so every pixel it reports lives in the same world frame as the arm. The framework is a durable lab asset, a public and documented multimodal dataset with a synchronized recording pipeline ready for the real workcell.

2. The gap

When a learned manipulation policy fails, the best way to recover depends on *what kind* of failure occurred, yet current systems hard-code a single recovery behavior. Recovery is splintered into siloed lines that each commit to one mechanism, whether retry and rewind (RaC, SPR), local-RL recovery (RecoveryChaining), VLM re-planning (FailSafe, SC-VLA), or human hand-off (the Cornell HITL framework, HRI 2026). Each works only in the regime it assumes. A rewind needs an in-distribution target to return to, local RL needs a reachable funnel, and re-planning breaks down once perception itself is out of distribution. To our knowledge, no single system does all three at once. It would have to type failures by their relation to a policy's *competence region*, learn a meta-policy that *selects* among three or more heterogeneous recovery mechanisms (not binary act-versus-ask or act-think-abstain routing) under explicit cost and risk constraints, and evaluate on real-robot rollouts with monolithic end-to-end policies. Damaged-can manipulation is an ideal testbed, since a dented or bulged can has different grasp affordances and failure modes than a nominal one, so failures are frequent, varied, and safety-relevant.

3. Contribution

1. **A failure taxonomy grounded in competence, not error labels.** Each failure is typed by how far the current state lies outside the region where the frozen policy is reliable, read from two signals (latent-state density and small-ensemble action disagreement) with a control-invariant safe set as a hard floor. The state's position fixes the recovery, in-region to retry, near the boundary to rewind and re-approach, outside but plannable to re-plan, and outside or risky to ask a human, the terminal fallback a person always resolves. A control-invariant safe set sits beneath these as a hard floor, halting the arm if a state turns genuinely unsafe, independent of the selector.
2. **A learned selector over the recovery moves.** Given a failure, its features, the competence signals, and task context, a cost-sensitive selector picks one of four recovery options, or *arms* (retry, rewind and re-approach, re-plan, or ask a human), under an explicit Lagrangian budget on the ask-human arm, which also serves as the terminal safety fallback. This unifies four lines of work that each hard-code a single move.
3. **Counterfactual recovery evaluation.** For each injected failure, reset and replay all four arms

and record each one's cost (time, human-intervention effort, safety violations). This yields the per-failure oracle and a new metric, *recovery-regret* (realized cost minus oracle cost), replacing binary recovery-success.

4. Method and plan

Part 1, HARVEST framework (robot-free). Build the episode schema, the synchronized multi-stream recorder with timestamp-consistency verification, the episode-protocol state machine, the physics-sim backend, rosbag2 input and output (via the pure-Python `rosbags` library, so no ROS 2 toolchain is needed on the development machine while the format stays lab-native), annotation, and by-can dataset packaging, all under strict test-driven development. The deliverable is a tested framework that generates and packages multimodal episodes in simulation and is ready to run on the real Kinova by swapping the simulator backend for the driver backend.

Part 2, recovery (on top). On a frozen base policy (inference or LoRA only), build the competence model, inject 100–150 scripted failures across the can-manipulation tasks, and replay the full four-arm grid per failure to obtain the per-failure oracle and recovery-regret. The *make-or-break gate* runs in simulation, before any robot commitment. It is a GO if two things hold, that the per-failure oracle beats the best fixed strategy by at least 15% recovery-regret, which shows there is headroom for a selector to capture, and that a linear probe separates the failure classes above about 70%, which shows that headroom is learnable from the competence signals. A NO-GO still ships a benchmark plus the recovery-regret metric. Then train and calibrate the selector, targeting 400–600 labeled instances drawn from the scripted injections together with the natural failures that arise across HARVEST collection. Because those failures arise during collection anyway, the recovery study adds no extra data collection and no change to the timeline.

5. Feasibility and resources

The framework is built robot-free so that no time is lost while hardware access is arranged, which is currently pending coordination. Physical collection on the real Kinova, across hundreds of cans, is the primary deliverable and begins as soon as the arm is available. There is no policy training. A frozen base policy plus a small selector head and competence calibration fit on a single strong GPU such as an H200 (inference or LoRA only). The simulation pilot doubles as the training set, so the make-or-break gate carries minimal downside before any robot commitment. The one robot-coupled stage needs scripted-failure time on the real workcell with repeatable resets, coordinated through the lab.

6. Venue

The HARVEST dataset and framework target IEEE Robotics and Automation Letters (RA-L). The recovery-selection contribution targets CoRL 2027, with an ICRA workshop as an on-ramp. The recovery-regret metric and the failure benchmark stand as defensible contributions even if a learned selector appears elsewhere first.

Key related work (citations to finalize). The damaged-can safety criteria come from USDA and CDC food-safety guidance on shelf-stable foods and botulism. The recovery lines I build on and compare against are Back-to-the-Manifold (state-density recovery), control-invariant safe sets, RaC and SPR (retry and rewind), RecoveryChaining (local-RL recovery), FailSafe and SC-VLA (VLM re-planning), and the Cornell human-in-the-loop work (Banerjee et al., HRI 2026). The base policy is ACT (Zhao et al. 2023).